Laboratorio · Sockets en Java

Sistemas Distribuidos · 61TI · 615000735 · 4.º curso · Grado en Tecnologías para la Sociedad de la Información

ETSISI – UPM · Curso 2026-27 · Aula 3203 · martes 22 de septiembre de 2026 · Práctica 2: el servidor de eco monohilo

Gustavo Hernández-Peñaloza y Javier Serrano (2024) · revisión de Carlos Castilla Ruiz y Guillermo Iglesias Hernández (2025)

Edición 2026-27 en castellano: David Díaz Pardo de Vera · david.diazp@upm.es · Despacho 4307

Basado en las transparencias 4-Sockets (Sockets in Java): Hernández-Peñaloza y Serrano (2024), revisadas por Castilla e Iglesias (2025), según las portadas de las ediciones de 2024 y 2025, y las propiedades del fichero de 2025. En Moodle, «Materiales de cursos anteriores».

Clase de laboratorio · martes 22 de septiembre · 19:00–21:00

Segunda parte de las transparencias; después, la práctica 2 en vuestros portátiles

Transparencias en castellano; cada término técnico lleva su forma inglesa entre paréntesis la primera vez que aparece, así: puerto (ingl. *port*). «Socket» se conserva en inglés, como en toda la documentación. Los nombres de clases y de métodos, y las palabras reservadas de Java, se escriben tal cual. Glosario del curso: SD_Glosario_2627.

Cómo se organiza esta parte

Media hora escasa de transparencias y, después, la práctica 2 en vuestros portátiles.

§	Sección	Minutos
1	Qué es un socket: TCP, UDP, puertos y la API de Java	5
2	Una conexión TCP, paso a paso: `ServerSocket`, `accept`, `Socket`	9
3	El servidor de eco: el bucle de líneas, lo que viene y la práctica 2	11
4	UDP, en una pasada	en casa
5	Para ampliar: enlaces y bibliografía	en casa

En Moodle: el enunciado de la práctica 2 (ES y EN), este PDF, el de entrada y salida y el glosario del curso.

¿Qué es un socket, y qué hay debajo?

Cinco minutos

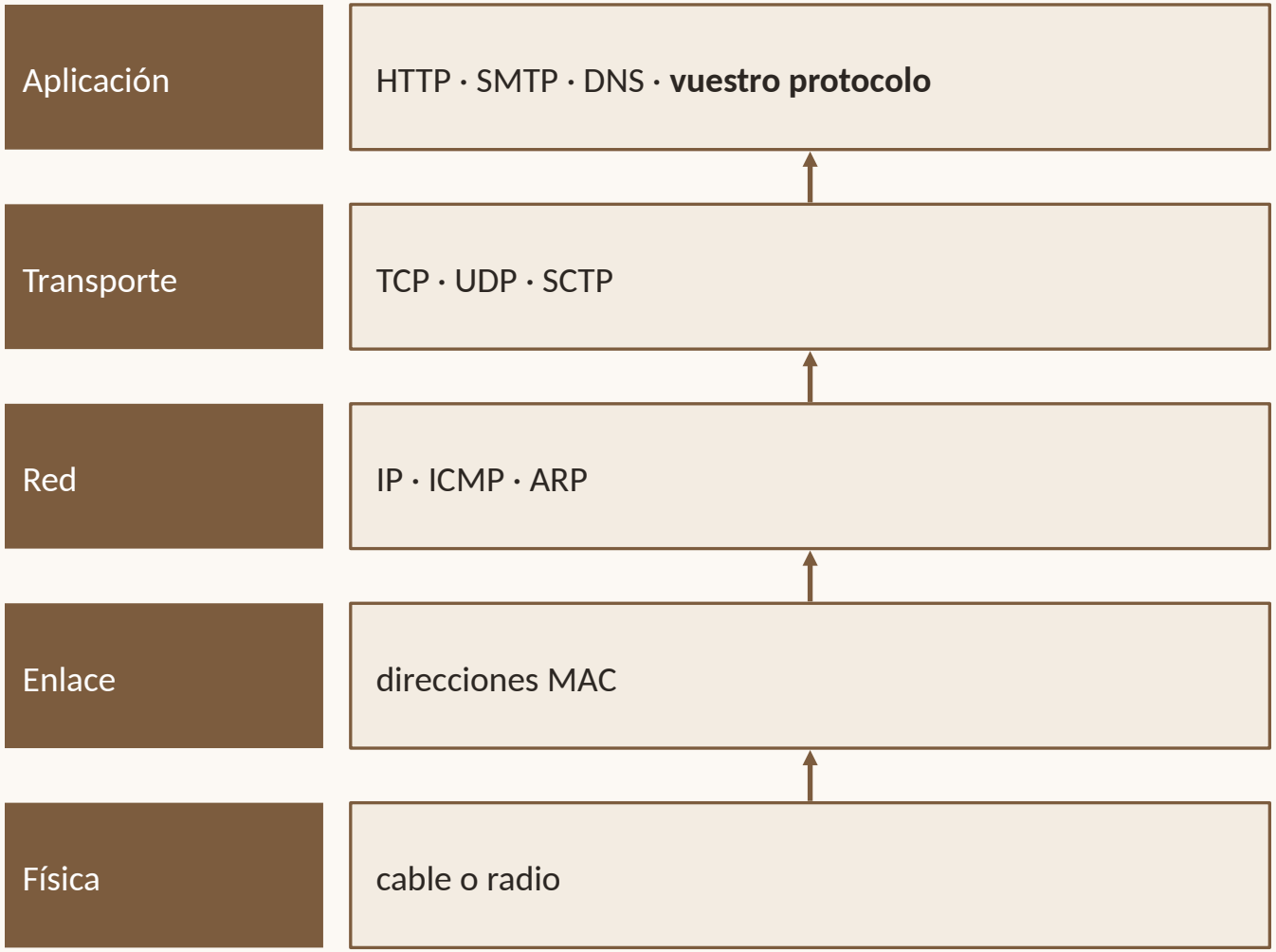
Socket: un extremo de la conversación

- **Socket**: extremo de una comunicación de red entre dos procesos
- En TCP, ofrece un flujo de bytes en cada sentido
- Su dirección local es una IP y un puerto (ingl. *port*): `138.100.55.4:9999`
- Una conexión TCP queda identificada por sus dos extremos

> La interfaz de programación del paso de mensajes, y la única que usa el laboratorio.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 4

Dónde vive el socket



Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 5 (figura redibujada)

- El socket es la puerta entre vuestro código y el transporte
- Por debajo, el sistema operativo y la red
- Por encima, un protocolo que escribís vosotros
- El eco de hoy y el banco de la práctica 4 viven en la capa de arriba

TCP y UDP: lo que promete cada uno

	TCP	UDP
Conexión	sí, se establece antes	no
Alcance	uno a uno	uno a uno, uno a varios
Pérdidas	no: retransmite	sí, sin aviso
Duplicados	no	posibles
Orden	el de envío (FIFO)	cualquiera
Ejemplos	web, correo, SSH	juego en red, voz, DNS

Pérdidas y duplicados: dos de las tres propiedades del enlace fiable. → Tema 2

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 6

Las clases de Java

Clase	Tipo	Papel
`java.net.Socket`	flujo	cliente TCP
`java.net.ServerSocket`	flujo	servidor TCP
`java.net.DatagramSocket`	datagrama (ingl. *datagram*)	cliente y servidor UDP
`javax.net.ssl.SSLSocket`	flujo	cliente TCP cifrado

El laboratorio usa las dos primeras de principio a fin.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 7 (nombres de paquete corregidos)

Puertos: quién puede usar cuáles

Intervalo	Grupo	Ejemplos
0 a 1023	bien conocidos	22 SSH · 25 SMTP · 80 HTTP · 443 HTTPS
1024 a 49151	registrados	3306 MySQL · 5432 PostgreSQL · 8080 HTTP alternativo
49152 a 65535	privados y dinámicos	sin servicios asignados

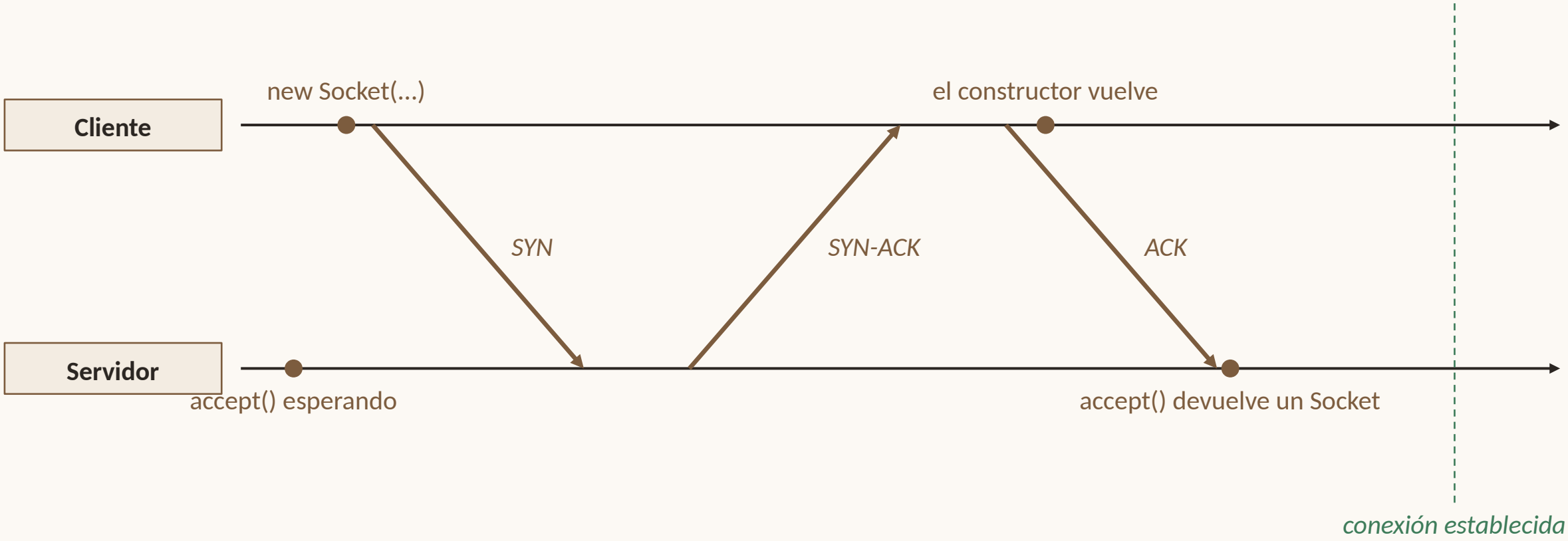
Las prácticas usan el 9999. En Linux y macOS, por debajo de 1024 hacen falta privilegios de administrador.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 9 (dos puertos corregidos; registro IANA)

¿Qué ocurre, y en qué orden, cuando un cliente se conecta?

Nueve minutos

Primero, el saludo a tres vías (ingl. *three-way handshake*)

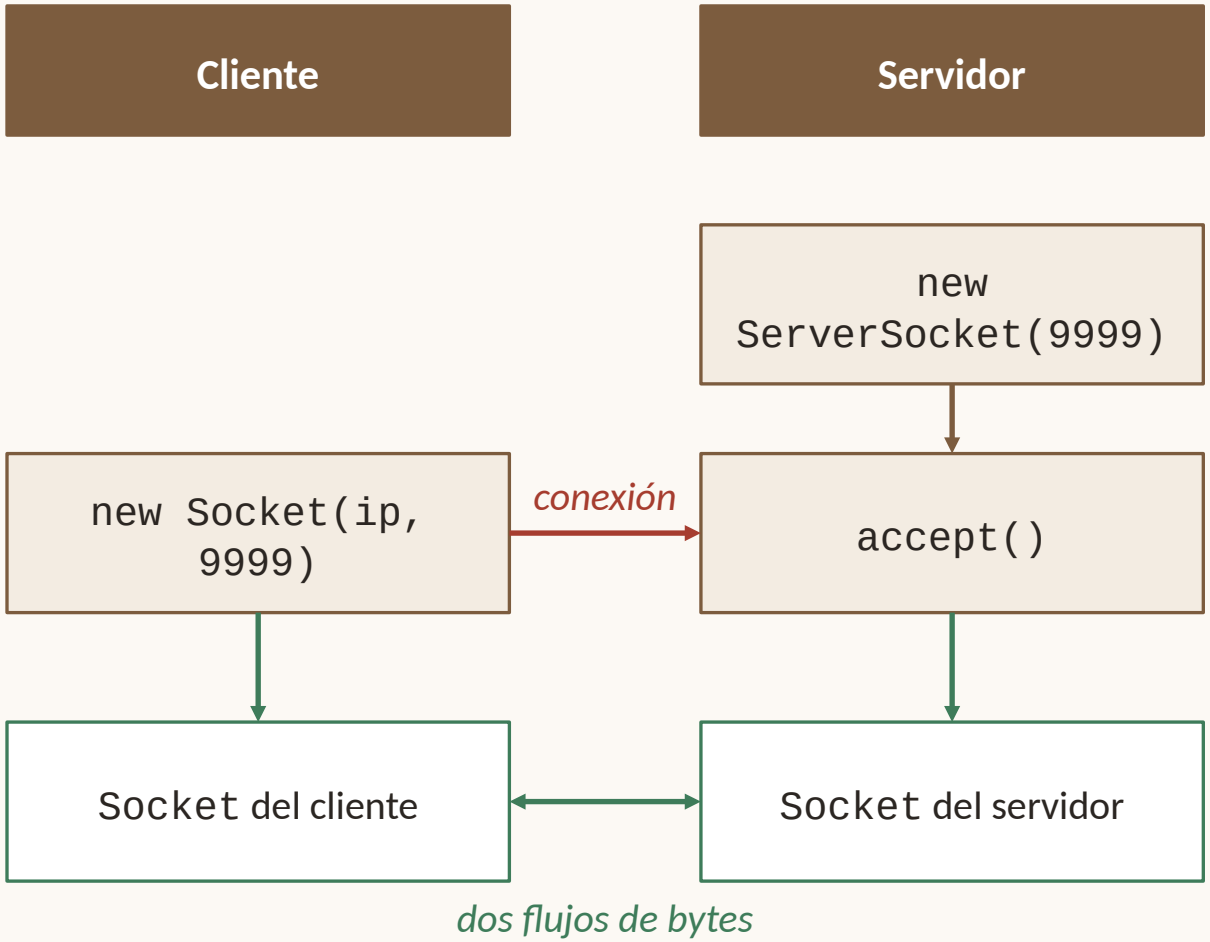


Lo hacen los dos sistemas operativos. Vuestro código solo ve el principio y el final.

Y lo que queda es lo de siempre: un flujo de entrada y otro de salida.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 11 (figura redibujada; SYN corregido)

Quién llama a qué



- El servidor abre un `ServerSocket` y espera en `accept()`
- `accept()` se bloquea hasta que entra una conexión
- Cada cliente aceptado produce un `Socket` nuevo en el servidor
- El `ServerSocket` sigue escuchando; el `Socket` es el que habla

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 8 y 12 (figura redibujada)

El servidor, hasta la conexión

Abrir, aceptar, envolver: ya hay dos flujos de bytes.

```
1  ServerSocket socketServidor = new ServerSocket(9999);
2  Socket socket = socketServidor.accept();   // aquí se bloquea
3  BufferedReader entrada = new BufferedReader(
4          new InputStreamReader(socket.getInputStream()));
5  PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);
```

El `true` de `PrintWriter` es el autovaciado: cada `println` sale al cable sin esperar.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 8 y 15 (listado nuevo sobre el esquema heredado, compilado con JDK 21)

El cliente, hasta la conexión

```
1  Socket socket = new Socket("localhost", 9999);
2  PrintWriter alServidor = new PrintWriter(socket.getOutputStream(), true);
3  BufferedReader delServidor = new BufferedReader(
4          new InputStreamReader(socket.getInputStream()));
```

Simétrico al servidor, salvo la primera línea: el cliente sabe a quién llama.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 14 (listado nuevo sobre el esquema heredado, compilado con JDK 21)

Cerrar: quién cierra qué

Un `close()` de menos deja dos programas esperándose.

- El cliente cierra su `Socket` → el servidor lee `null`
- El servidor cierra el `Socket` de ese cliente cuando la conversación acaba
- El `ServerSocket` se cierra al final de todo, y deja de admitir clientes
- Cerrar el `ServerSocket` no corta las conversaciones ya abiertas

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 13 (paso 5 corregido)

¿Cómo es el servidor más sencillo que se puede escribir?

Once minutos

El bucle de eco, entero

Un cliente cada vez, líneas de texto, hasta que el cliente cierre.

```
 1  ServerSocket socketServidor = new ServerSocket(9999);
 2  Socket socket = socketServidor.accept();
 3  BufferedReader entrada = new BufferedReader(
 4          new InputStreamReader(socket.getInputStream()));
 5  PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);
 6
 7  String linea = entrada.readLine();
 8  while (linea != null) {            // null = el cliente ha cerrado
 9      salida.println(linea);         // el eco
10      linea = entrada.readLine();
11  }
12  socket.close();
13  socketServidor.close();
```

Del `null` no se hace eco: termina el bucle.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 21 (enunciado del ejercicio 1); listado nuevo de la edición 2026-27, compilado y ejecutado con JDK 21

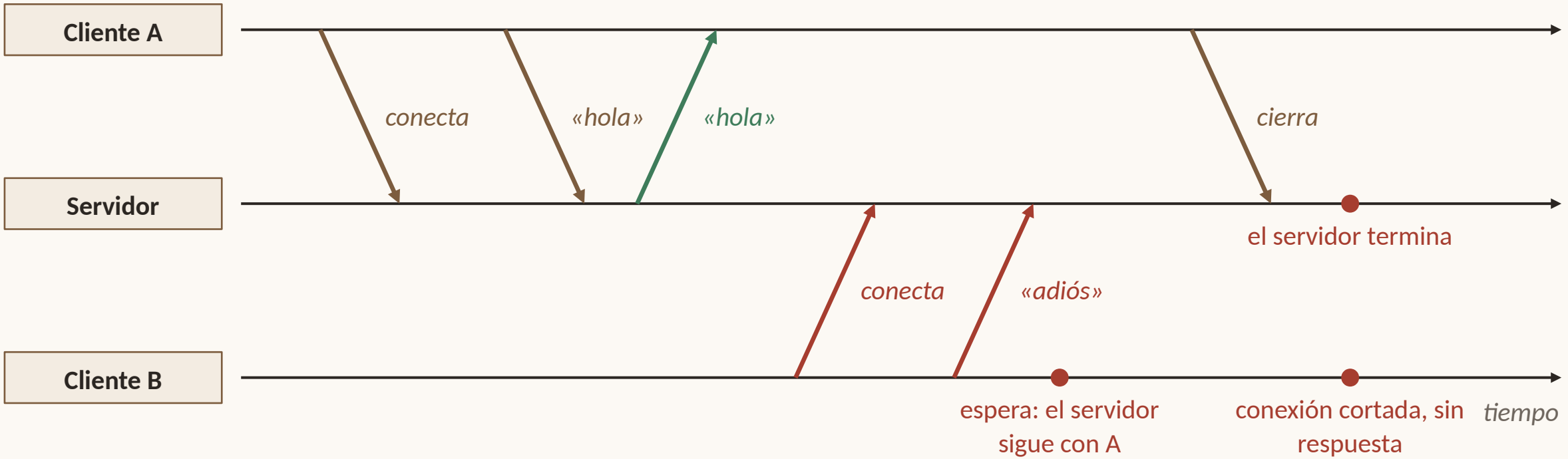
Dos formas de que no funcione

Ocurren todos los años, y ninguna dice dónde está el fallo.

- Sin autovaciado: el servidor escribe, el cliente espera, nadie se mueve
- Con eco del `null`: el bucle del servidor pierde su condición de salida
- La primera se arregla con `new PrintWriter(flujo, true)` o con `flush()`
- La segunda, comprobando el `null` antes de hacer el eco

Fuentes: Enunciado de la práctica 2 (nota sobre el fin de fichero); defectos observados en cursos anteriores

Un servidor monohilo con dos clientes

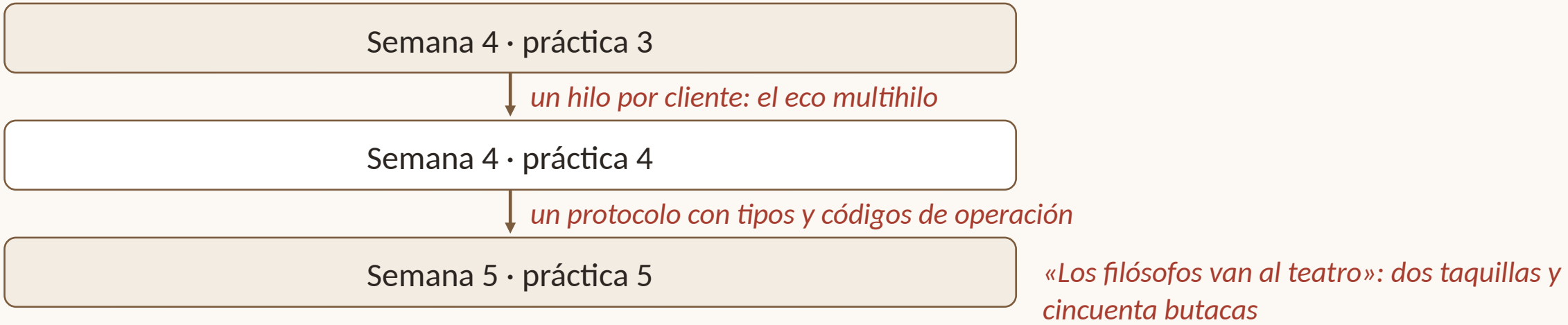


B conecta sin problemas. No recibe nada: el único hilo del servidor sigue con A.

Y cuando A cierra, este servidor termina: B se queda sin respuesta.

Fuentes: Figura nueva de la edición 2026-27; comportamiento medido con JDK 21

Adónde lleva esto



- El servidor de hoy, con más gente dentro y algo que repartir

Fuentes: Hoja de ruta del laboratorio 2026-27 (Moodle)

Lo que hay que saber hacer

1. Abrir un `ServerSocket`, esperar en `accept()` y obtener un `Socket`
2. Conectar un cliente con `new Socket(dirección, puerto)`
3. Montar la cadena de lectura de líneas y un `PrintWriter` con autovaciado sobre un socket
4. Escribir el bucle de eco y terminarlo bien cuando llega el `null`
5. Decir por qué un servidor monohilo deja esperando al segundo cliente

Fuentes: Enunciado de la práctica 2 · Guía de aprendizaje 2026-27: PR-EX-1, «Java. TCP y Marshalling/Unmarshalling». En este curso, prácticas 0 a 5

La práctica 2: lo que hay que escribir

a) Un cliente que lee líneas del teclado y las manda por el socket

b) …e imprime lo que el servidor devuelve

c) Un servidor de eco monohilo en `localhost:9999`, un solo cliente

d) Los dos programas terminan solos cuando el usuario teclea el fin de fichero

e) Entrega voluntaria por Moodle: los dos ficheros `.java` y las respuestas

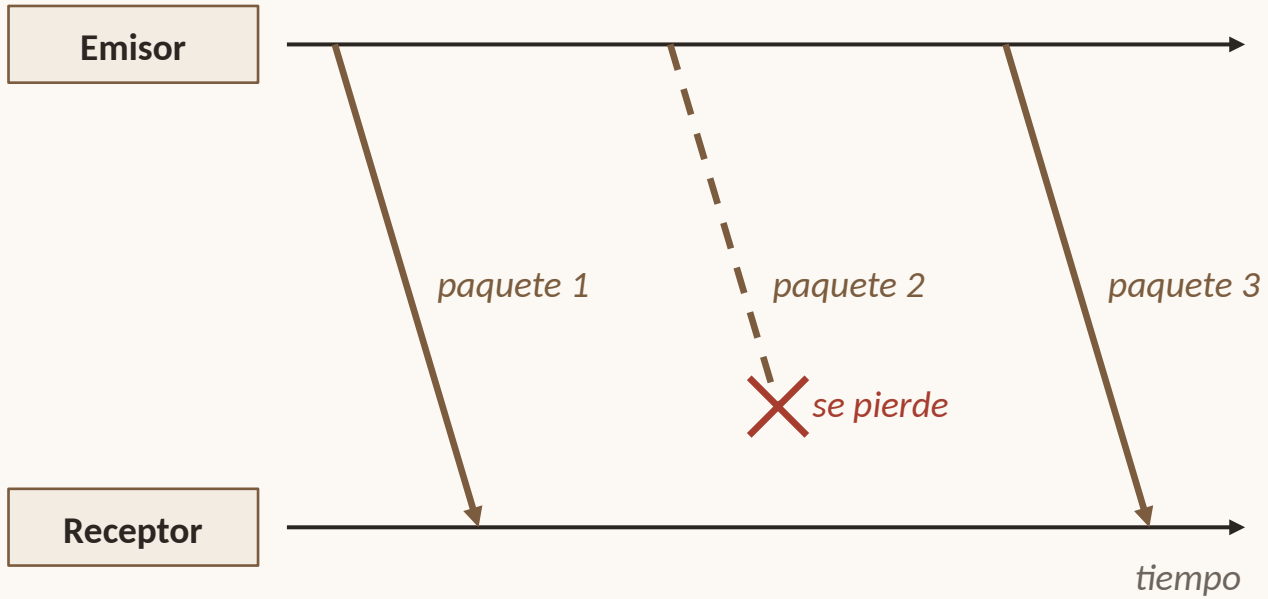
Fuentes: Enunciado de la práctica 2: SD_Prac2_EcoMonohilo_2627 (ES) · SD_Prac2_MonothreadEcho_2627 (EN)

¿Y si no hace falta una conexión?

Para leer en casa

UDP: paquetes sueltos, sin conexión

- Cada datagrama viaja solo, con la dirección dentro
- Sin conexión, sin retransmisión, sin orden garantizado
- `DatagramSocket` para enviar y recibir · `DatagramPacket` para el contenido
- Más ligero: sirve cuando llegar tarde es peor que no llegar



Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 16, 18–20 (figura nueva)

Cuál elegir

Si necesitáis…	Elegid	Porque
que llegue todo y en orden	TCP	lo garantiza el transporte, no vuestro código
mandar lo mismo a un grupo	UDP	un datagrama puede ir a varios destinatarios
latencia mínima	UDP	sin retransmisiones ni reordenación
lo del laboratorio	TCP	todas las prácticas y los dos exámenes

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 6 y 16 (tabla nueva)

¿Y a partir de aquí?

Para leer en casa

Para ampliar

Recurso	Dónde
Oracle, API de Java 21: `Socket` y `ServerSocket`	docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/Socket.html
Oracle, The Java Tutorials: Custom Networking (sockets)	docs.oracle.com/javase/tutorial/networking/sockets/
IANA, registro de nombres de servicio y números de puerto	www.iana.org/assignments/service-names-port-numbers/
M. Ben-Ari, *Principles of Concurrent and Distributed Programming*, Pearson, 2006	libro de referencia; vista previa en Google Libros

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 24 (enlaces actualizados a Java 21)